Lecture 24: Cryptography and Security

From primitives to TLS

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Today

Session 1 — Primitives

- Threat model: α, β, τ
- Symmetric crypto (AES, GCM)
- Hash functions (SHA-256)
- **RSA math, worked end to end**
- Diffie–Hellman

Session 2 — Protocols

- MACs and signatures
- PKI: certificates and CAs
- TLS 1.3 handshake
- Closing the loop (QUIC / L23)
- Course wrap-up

Theme

Last week you saw that every protocol ran in plaintext on the wire. Today we fix that.

Key Acronyms

Acronym	Meaning
ACME	Automatic Cert. Mgmt. Env.
AES	Advanced Encryption Standard
AEAD	Auth. Encryption w/ Assoc. Data
CA	Certificate Authority
CBC	Cipher Block Chaining
DH	Diffie–Hellman
ECDSA	Elliptic Curve Digital Sig. Alg.
ECB	Electronic Codebook
FIPS	Federal Info. Processing Standard
GCM	Galois/Counter Mode

Acronym	Meaning
HKDF	HMAC-based Key Derivation Fn.
HMAC	Hash-based Message Auth. Code
MAC	Message Authentication Code[†]
NIST	Nat'l Inst. of Standards & Tech.
PKI	Public Key Infrastructure
PSK	Pre-Shared Key[‡]
QUIC	Quick UDP Internet Connections
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
TLS	Transport Layer Security

[†] *MAC = Media Access Control in L7/L8 (link layer).*

[‡] *PSK = Phase Shift Keying in L3/L4 (physical layer).*

Three Actors

Throughout this lecture:

α	Sender ("alpha")
β	Receiver ("beta")
τ	Adversary on the path ("tau")

Every cryptographic story we tell today has these three characters. τ is what makes it interesting.

What Can τ Do?

Attack	Threat	We need
Eavesdrop	read the message	**confidentiality**
Modify	change bits mid-flight	**integrity**
Impersonate	pretend to be the peer	**authentication**
Replay	resend old captured msg	**freshness**
Deny later	"I never sent that"	**non-repudiation**

Building security is addressing these threats one at a time. Each has a matching primitive. This table is the spine of the lecture.

Symmetric Cryptography

α and β share a secret key k:

$$c = E_k(m) \qquad m = D_k(c)$$

Workhorse: AES (NIST FIPS 197, 2001).

- 128-bit blocks; 128 / 192 / 256-bit keys.
- Hardware-accelerated on every modern CPU (AES-NI).
- $\sim$5 GB/s single-core — essentially free.

Block cipher modes:

- **ECB** — leaks patterns. *Never use.*
- **CBC** — confidentiality only, no integrity. Legacy.
- **GCM** — authenticated encryption (AEAD). Confidentiality **and** integrity in one primitive. **What TLS 1.3 uses.**

The Key Distribution Problem

Symmetric crypto needs a shared key. How do α and β agree on k when τ is watching the channel?

Three answers we will build to:

1. **Pre-shared keys.** Works. Doesn't scale.
2. **Asymmetric crypto to transport k.** The classic TLS 1.2 pattern.
3. **Diffie–Hellman key agreement.** The TLS 1.3 pattern — and the interesting one.

Before we can do either asymmetric answer, we need public-key cryptography — coming up in a few slides.

Hash Functions

A hash function $h : \{0,1\}^* \to \{0,1\}^n$ maps arbitrary-length input to a fixed-length digest.

SHA-256: 256-bit digest. No key. One-way compression.

Three security properties:

1. **Preimage resistance.** Given $y = h(m)$, hard to find any m' with $h(m') = y$.
2. **Second-preimage resistance.** Given m, hard to find $m' \neq m$ with $h(m') = h(m)$.
3. **Collision resistance.** Hard to find any $m \neq m'$ with $h(m) = h(m')$.

Uses: file integrity, password storage, commitment schemes, building blocks for MACs and signatures (coming up).

See: `demo_hash_124.py` — avalanche effect, commitment.

The Asymmetric Idea

Each party holds a **key pair**: $(K_{\text{pub}}, K_{\text{priv}})$.

- Knowing the public key does *not* reveal the private key.
- **Encrypt** with *receiver's* public key, decrypt with their private key $\rightarrow$ confidentiality.
- **Sign** with *sender's* private key, verify with their public key $\rightarrow$ authentication, integrity, non-repudiation.

Two operations that break everything else open. We spend the next 15 minutes on one specific construction: **RSA**.

Number Theory We Need

Two classical results.

Euler's totient. $\varphi(n) =$ count of integers in $[1, n]$ coprime to n. For distinct primes p, q:

$$\varphi(pq) = (p-1)(q-1).$$

Euler's theorem. If $\gcd(a, n) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

That's it. RSA is a three-line corollary.

RSA Key Generation

1. Pick two large primes p, q. (1024 bits each, in practice.)
2. Compute $n = pq$ and $\varphi(n) = (p-1)(q-1)$.
3. Pick public exponent e with $\gcd(e, \varphi(n)) = 1$. In practice $e = 65537$.
4. Compute $d = e^{-1} \pmod{\varphi(n)}$ by the extended Euclidean algorithm.

Public key	(n, e)
Private key	(n, d)

RSA Encryption and Decryption

Encode the message as an integer m with $0 \leq m < n$.

$$\begin{aligned} \text{Encrypt:} \quad & c = m^e \bmod n \\ \text{Decrypt:} \quad & m = c^d \bmod n \end{aligned}$$

Why this works. We chose $ed \equiv 1 \pmod{\varphi(n)}$, so $ed = 1 + k\varphi(n)$ for some integer k. Then:

$$c^d = m^{ed} = m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k \equiv m \pmod{n}$$

by Euler's theorem.

Worked Example

Key generation.

- $p = 11$, $q = 13 \quad \Rightarrow \quad n = 143$, $\varphi(n) = 10 \cdot 12 = 120$.
- $e = 7$ (coprime to 120).
- $d = 103$ (check: $7 \cdot 103 = 721 = 6 \cdot 120 + 1$).

Encrypt $m = 9$:

$$c = 9^7 \bmod 143 = 4{,}782{,}969 \bmod 143 = 48.$$

Decrypt $c = 48$:

$$m = 48^{103} \bmod 143 = 9.$$

See: `demo_rsa_math_124.py` — run it, change numbers.

Why RSA Is Secure

An attacker with (n, e) who wants d needs $\varphi(n) = (p-1)(q-1)$, which needs p and q — i.e., must **factor** n.

- Best classical algorithms for a 2048-bit semiprime: $\sim 2^{112}$ operations. Beyond reach.
- Shor's algorithm on a large enough quantum computer: polynomial time. Hence *post-quantum cryptography* — an active area.

The hardness assumption

RSA security reduces to: factoring is hard. That's the entire bet.

Signing Is the Same Math, Flipped

For RSA:

$$\text{Sign:} \quad s = h(m)^d \bmod n \quad \text{(signer uses } \textit{private} \text{ exponent)}$$

$$\text{Verify:} \quad s^e \bmod n \stackrel{?}{=} h(m) \quad \text{(verifier uses } \textit{public} \text{ exponent)}$$

Why we hash first.

- Efficiency: one modular exponentiation signs arbitrary-length input.
- Security: textbook RSA on raw m has known forgery attacks (multiplicativity). Hash + padding (RSA-PSS) avoids them.

Diffie–Hellman Key Exchange

Public parameters: a large prime p and generator g. Anyone can know these.

Protocol:

1. α picks private a, sends $A = g^a \bmod p$.
2. β picks private b, sends $B = g^b \bmod p$.
3. α computes $B^a = g^{ab} \bmod p$.
4. β computes $A^b = g^{ab} \bmod p$.

Now α and β share $g^{ab} \bmod p$ *without ever sending it*.

τ sees g^a and g^b on the wire. To recover g^{ab}, τ must solve the **discrete log problem** — believed hard.

Why It Matters for TLS 1.3

Every modern TLS connection uses *ephemeral* Diffie–Hellman (usually on an elliptic curve).

β's long-term RSA/ECDSA key is used only to *sign* β's DH public share — **not** to transport the session key.

Forward Secrecy

If β's long-term private key leaks tomorrow, recorded traffic from today is **still safe** — the session key was derived from ephemeral values that were discarded.
TLS 1.3 removed the old RSA key-transport mode entirely to guarantee this property.

Primitive to Requirement

Requirement	Primitive	Key kind
Confidentiality	AES-GCM	symmetric
Integrity	MAC or AEAD	symmetric
Authentication (peer)	signature or MAC	asym or sym
Authentication (message)	MAC	symmetric
Non-repudiation	**signature**	**asymmetric only**
Freshness	nonce / timestamp	protocol

Why signatures for non-repudiation? α and β both know the MAC key — either could have made the MAC. A private key is held by exactly one party, so a valid signature proves *which* party made it, verifiable by any third party.

MACs — Message Authentication Codes

Given a shared key k, prove:

- The message has not been modified (integrity).
- It came from someone who knows k (authentication).

HMAC (RFC 2104):

$$\mathrm{HMAC}_k(m) = h\big((k \oplus \mathrm{opad}) \,\|\, h((k \oplus \mathrm{ipad}) \,\|\, m)\big)$$

The two-level hash defends against attacks on naive $h(k\|m)$.

What HMAC does not give you: non-repudiation. Both α and β have k; either could have made the tag.

In modern TLS, MACs are subsumed by AEAD (AES-GCM), which rolls encryption and MAC into one primitive.

Digital Signatures

$$s = \text{Sign}_{K_{\text{priv}}}(h(m)) \qquad \text{Verify}_{K_{\text{pub}}}(s, h(m)) \in \{T, F\}$$

- Integrity: τ cannot change m without breaking verification.
- Authentication: only K_{priv} holder could have signed.
- **Non-repudiation:** K_{priv} holder can't deny it.

Sign the hash, not the message. Efficiency + security (padding schemes like RSA-PSS avoid textbook-RSA forgery attacks).

Modern alternatives. Ed25519 and ECDSA are much smaller than RSA for equivalent security:

- Ed25519 signature: $\sim$64 bytes
- RSA-2048 signature: 256 bytes

Most new systems default to Ed25519.

The Trust Problem

β's browser just received a public key claiming to be from `bank.com`. How does β know τ didn't generate that public key?

A public key, by itself, is just bits. It says nothing about *whose* key it is.

Answer: certificates. A trusted third party cryptographically binds the public key to an identity.

Certificates and CAs

A **certificate**:

$$\text{cert} = (\text{identity}, K_{\text{pub}}, \text{validity}, \ldots) + \text{signature by a CA}$$

A **Certificate Authority (CA)** is a trusted third party. Browsers and OSes ship a **root store** — a preinstalled list of root CA public keys.

Chain of trust:

$$\text{leaf cert} \xrightarrow{\text{signed by}} \text{intermediate CA} \xrightarrow{\text{signed by}} \text{root CA}$$

β's browser walks the chain from leaf to any root in its store.

- **Roots:** hundreds globally.
- **Intermediates:** many.
- **Leaves:** one per domain.

Real-World PKI

Let's Encrypt changed everything.

- Free, automated (ACME protocol), launched 2016.
- HTTPS share of web traffic: ∼30% in 2015 → ∼95% today.
- Made "HTTPS by default" economically obvious.

When CAs go bad:

- **DigiNotar (2011)** — compromised, issued fraudulent `*.google.com` certs, removed from every root store. Company bankrupt within weeks.
- **Symantec (2017–18)** — Google progressively distrusted Symantec's CA hierarchy after mis-issuance.

Certificate pinning — apps "pin" to a specific expected cert or public key. Defense in depth at the cost of operational fragility.

Live: Inspect a Real Cert Chain

```
openssl s_client -connect bu.edu:443 -servername bu.edu < /dev/null
openssl s_client -connect expired.badssl.com:443 \
    -servername expired.badssl.com
```

What to look for in the output:

- The `depth` counter: 0 = leaf cert; higher = intermediate CAs; up to a root in your local trust store.
- `subject` and `issuer` lines — who it's for, who signed it.
- `Server Temp Key` — the ephemeral DH share for this session (forward secrecy in action).
- `Cipher: TLS_AES_256_GCM_SHA384` — AES-GCM + SHA-384-based KDF. All L24 primitives in one line.

See: `demo_openssl_client_l24.sh`

TLS 1.3 Handshake

1. **ClientHello.** α sends supported ciphers and an ephemeral DH public share g^a.
2. **ServerHello.** β picks a cipher and sends:
   - its own ephemeral DH share g^b,
   - its certificate chain,
   - a signature over the handshake transcript.
3. α **verifies:** cert chain against root store, then β's signature.
4. **Both derive session keys.** Compute g^{ab}, run through HKDF, get AES-GCM keys.
5. **Application data.** Encrypted with AES-GCM.

One round-trip. Every primitive from this lecture used once.

Every Primitive, One Handshake

Step	Primitive	From
Key share	Diffie–Hellman (ephemeral)	§DH
Cert chain verify	Digital signatures + PKI	§Signatures, PKI
Handshake auth	Signature on transcript	§Signatures
Bulk encryption	AES-GCM (AEAD)	§Symmetric
Bulk integrity	AES-GCM (AEAD)	§Symmetric
Key derivation	HKDF (hash-based)	§Hash

Asymmetric crypto used once (at the handshake, tens of ms). **Symmetric crypto everywhere else** (Gb/s, essentially free). That division of labor is the whole design.

Forward Secrecy, Revisited

Session keys come from *ephemeral* DH values that are discarded at the end of the session.

Consequence: even if β's long-term signing key is compromised tomorrow, traffic recorded today cannot be decrypted.

The long-term key only signs handshakes — it never encrypts anything.

Store-now-decrypt-later

Adversaries are recording encrypted traffic today, betting that tomorrow's cryptography (or tomorrow's key compromise) will let them decrypt it. Forward secrecy is the defense.

Loop Back to L23: QUIC

Recall from L23: QUIC is HTTP/3's transport, over UDP, with TLS built in.

What "built in" means: QUIC integrates the TLS 1.3 handshake into the transport-layer handshake itself.

- One round-trip establishes connection **and** crypto.
- 0-RTT resumption sends application data in the first packet on return visits.
- Streams are encrypted independently — even packet headers are partially encrypted.

The promise from L23, delivered: QUIC closes the full loop. Every layer, encrypted by default, in one round-trip.

The Course in Three Sentences

1. **L1–L9.** What bits mean, how they travel, how they survive noise, how a local network works.
2. **L13–L22.** How packets get across the world, how reliability is built on best-effort, and the tools that let you see and touch it all.
3. **L23–L24.** What people actually build on top — HTTP, DNS, QUIC, TLS — and the cryptography that makes any of it safe.

A full stack, from Shannon to TLS, in one semester.

What to Study Next

Not on any exam. Just seeds.

- **IPv6** deployment and the long migration.
- **BGP** and the economics of internet routing.
- **SDN** — software-defined networking (OpenFlow, P4).
- **Net neutrality, surveillance, privacy-enhancing tech.**
- **Post-quantum cryptography** — the next decade of TLS.
- **High-throughput / low-latency systems** — DPDK, RDMA, kernel bypass.

You now have the vocabulary to read anything in any of these.

What's Next

L25 — Thu Apr 30 — Project Demo Day

Show us what you built.

Thanks for a great semester.